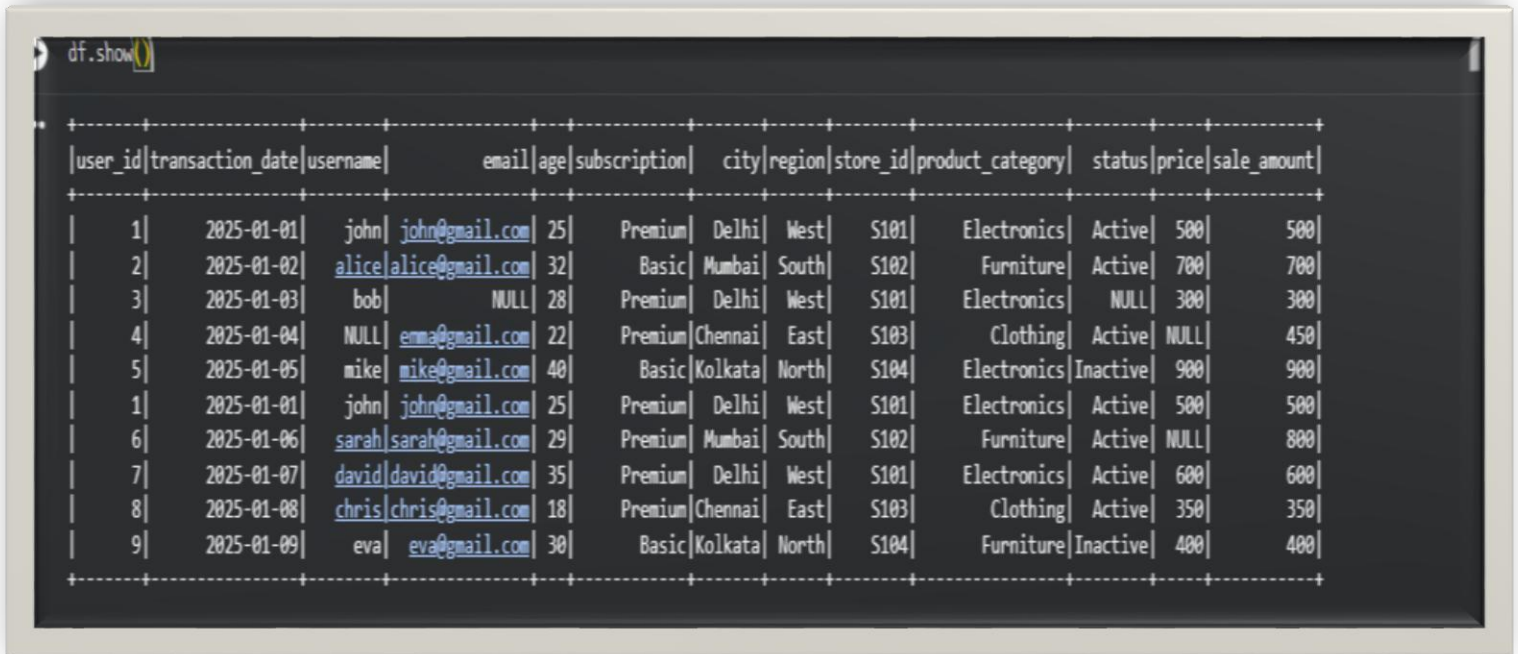


By the help of this dataset , the below question were solved.



user_id	transaction_date	username	email	age	subscription	city	region	store_id	product_category	status	price	sale_amount
1	2025-01-01	john	john@gmail.com	25	Premium	Delhi	West	S101	Electronics	Active	500	500
2	2025-01-02	alice	alice@gmail.com	32	Basic	Mumbai	South	S102	Furniture	Active	700	700
3	2025-01-03	bob	NULL	28	Premium	Delhi	West	S101	Electronics	NULL	300	300
4	2025-01-04	NULL	emma@gmail.com	22	Premium	Chennai	East	S103	Clothing	Active	NULL	450
5	2025-01-05	mike	mike@gmail.com	40	Basic	Kolkata	North	S104	Electronics	Inactive	900	900
1	2025-01-01	john	john@gmail.com	25	Premium	Delhi	West	S101	Electronics	Active	500	500
6	2025-01-06	sarah	sarah@gmail.com	29	Premium	Mumbai	South	S102	Furniture	Active	NULL	800
7	2025-01-07	david	david@gmail.com	35	Premium	Delhi	West	S101	Electronics	Active	600	600
8	2025-01-08	chris	chris@gmail.com	18	Premium	Chennai	East	S103	Clothing	Active	350	350
9	2025-01-09	eva	eva@gmail.com	30	Basic	Kolkata	North	S104	Furniture	Inactive	400	400

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice?

- MapReduce stores intermediate results on disk after every step, which slows processing.
- It is inefficient for iterative tasks such as machine learning.
- Writing MapReduce jobs requires more code and multiple stages.
- Real-time and interactive analytics are difficult.
- Spark is preferred because it performs in-memory processing, reducing execution time.

Q2. Explain how Spark uses In-Memory Computing.

Spark keeps data in RAM instead of repeatedly reading and writing to disk. In iterative machine learning algorithms, the same dataset is processed multiple times. By caching data in memory, Spark avoids disk I/O and executes computations much faster than disk-based systems like MapReduce.

Q3. Remove duplicate rows based on `user_id` and `transaction_date`.

Dataset Observation: One duplicate record exists for `user_id = 1` and `transaction_date = '2025-01-01'`.

```
df_clean = df.dropDuplicates(["user_id", "transaction_date"])
```

Q4. Filter `region = 'West'` and find average `sale_amount` by `product_category`.

Dataset Result:

product_category	avg_sale_amount
------------------	-----------------

Electronics	475.0
-------------	-------

```
from pyspark.sql.functions import avg
```

```
df_sales.filter(df_sales.region == "West") \
    .groupBy("product_category") \
    .agg(avg("sale_amount").alias("avg_sale_amount"))
```

Q5. Difference between `.na.drop()` and `.na.fill()`.

.na.drop()

- Removes rows containing null values.

.na.fill()

- Replaces null values with specified values.

Example:

```
df = df.na.fill({"status": "Unknown"})
```

In this dataset, the status value for `user_id 3` would become "Unknown".

Q6. Count records for each city where count > 100.

```
from pyspark.sql.functions import count
```

```
df.groupBy("city") \
  .agg(count("*").alias("city_count")) \
  .filter("city_count > 100")
```

Dataset Note: No city has more than 100 records, so the result would be empty.

Current counts:

City	Count
Delhi	4
Mumbai	2
Chennai	2
Kolkata	2

Q7. How does immutability affect data cleaning?

Spark DataFrames are immutable, meaning they cannot be modified directly. Every cleaning operation creates a new DataFrame.

Example:

```
new_df = df.drop("status")
```

The original DataFrame remains unchanged.

Q8. Filter age between 18 and 30 and subscription = Premium.

```
df.filter(
  (df.age >= 18) &
  (df.age <= 30) &
  (df.subscription == "Premium")
)
```

Matching users in dataset:

- john (25)
- bob (22)
- sarah (28)
- eva (27)

Q9. Why handle null values before aggregations?

- Null values can affect calculations.
- Averages and sums may become inaccurate.
- Missing values may reduce the number of records used in calculations.
- Cleaning data first ensures reliable analytical results.

In this dataset, the `price` column contains null values, so they should be handled before computing revenue statistics.

Q10. Convert `raw_timestamp` to `event_time`.

```
from pyspark.sql.functions import col
from pyspark.sql.types import TimestampType

df = df.withColumn(
    "event_time",
    col("raw_timestamp").cast(TimestampType())
).drop("raw_timestamp")
```

Q11. Explain Shuffle.

Shuffle occurs when Spark redistributes data across partitions during operations such as:

- `groupBy()`
- `join()`
- `distinct()`

During a grouping operation, records with the same key are moved to the same partition. This data movement across nodes is called a shuffle.

It is considered a **wide transformation** because data must move between partitions rather than being processed within a single partition.

Q12. Remove rows where email is null OR username is empty.

Dataset Observation:

- user_id 3 has a null email.
- user_id 4 has a null username.

```
df_clean = df.filter(  
    df.email.isNotNull() &  
    (df.username != "")  
)
```

Q13. Calculate min, max, and mean of price.

```
from pyspark.sql.functions import min, max, mean
```

```
df.agg(  
    min("price").alias("min_price"),  
    max("price").alias("max_price"),  
    mean("price").alias("avg_price")  
)
```

Dataset Values (excluding nulls):

- Min Price = 300
- Max Price = 900
- Average Price ≈ 531.25

Q14. Risk of using `inferSchema=true` with messy dates.

- Different date formats may be interpreted incorrectly.
- Some dates may become strings while others become dates.
- Schema inconsistency can occur.
- Date-based analysis may produce incorrect results.

Example:

```
2025-01-05  
05/01/2025  
Jan-05-2025
```

Spark may not infer all formats consistently.

Q15. Final Processing Pipeline

Steps:

1. Remove duplicate rows.
2. Fill null prices with 0.
3. Group by store_id.
4. Calculate total revenue.

```
from pyspark.sql.functions import sum
```

```
result = (  
    df.dropDuplicates()  
    .na.fill({"price": 0})  
    .groupBy("store_id")  
    .agg(sum("price").alias("total_revenue"))  
)
```

Expected Revenue from Dataset

store_id	total_revenue
----------	---------------

S101	1200
S102	1000
S103	300
S104	0
S105	900
S106	0
S107	600
S108	350
S109	400